

O's turn now, it will be player X's turn next. The alternative would have just said that if it's not player X's turn now, it will be player X's turn next. Even though `if` and `else` statements are a natural fit with Boolean conditions, the `PLAYER_X` and `PLAYER_O` values aren't the same as `True`, and `False`: `not PLAYER_X` is not the same as `PLAYER_O`. Therefore, it's helpful to be direct when checking the value of `playerTurn`.

Alternatively, I could have performed the same actions in a one-liner:

```
playerTurn = {PLAYER_X: PLAYER_O, PLAYER_O: PLAYER_X}[ playerTurn]
```

This line uses the dictionary trick mentioned in “Use Dictionaries Instead of a `switch` Statement” on page 101. But like many one-liners, it's not as readable as a direct `if` and `elif` statement.

Next, we define the `getNewBoard()` function:

```
def getNewBoard():
    """Returns a dictionary that represents a Four-in-a-Row board.

    The keys are (columnIndex, rowIndex) tuples of two integers, and the
    values are one of the "X", "O" or "." (empty space) strings."""
    board = {}
    for rowIndex in range(BOARD_HEIGHT):
        for columnIndex in range(BOARD_WIDTH):
            board[(columnIndex, rowIndex)] = EMPTY_SPACE
    return board
```

This function returns a dictionary that represents a Four-in-a-Row board. It has `(columnIndex, rowIndex)` tuples for keys (where `columnIndex` and `rowIndex` are integers), and the `'X'`, `'O'`, or `'.'` character for the tile at each place on the board. We store these strings in `PLAYER_X`, `PLAYER_O`, and `EMPTY_SPACE`, respectively.

Our Four-in-a-Row game is rather simple, so using a dictionary to represent the game board is a suitable technique. Still, we could have used an OOP approach instead. We'll explore OOP in Chapters 15 through 17.

The `displayBoard()` function takes a game board data structure for the `board` argument and displays the board onscreen using the `BOARD_TEMPLATE` constant: